

model/egmn.py (Q&A Documentation)

Q: What does this file implement at a high level? **A:** This file implements EGMN: a mixture-distribution watch-time predictor. It encodes sparse/sequence/continuous features, produces mixture parameters (mixture logits, exponential rate, Gaussian means/variances), and provides both a likelihood-based loss and a scalar prediction via the mixture mean.

Q: Why is a mixture distribution needed for watch-time? **A:** Watch-time has a heavy spike near 0 (quick-skip) and a long tail. A single regression head struggles to represent both. The model uses an Exponential component for the spike and multiple (truncated) Gaussians to represent the tail/modes.

Q: What are the model outputs and how are they used? **A:** Forward returns (pi, lambda, mu, sigma). Training uses them in the negative log-likelihood and auxiliary terms; inference uses them to compute a scalar expectation (mixture mean) as the final predicted watch-time.

Q: What does this file export (public API)? **A:** The public API is the set of classes/functions other modules import (sometimes re-exported by package initializers). The key top-level definitions detected here are:

```
class EGMN # methods: __init__, build, init, forward, loss, get_quantile, predict
```

Q: What are the main dependencies (imports) and why do they matter? **A:** Imports show whether this file is model code (torch), data code (pandas), or evaluation/analysis (sklearn). They also determine runtime requirements.

```
import torch
import torch.nn as nn
import torch.distributions as D
import torch.nn.functional as F
import numpy as np
```

```
from model.layers import FactorizationMachine, MultiLayerPerceptron, DurationMultiLayerPerceptron
```

```
class EGMN(torch.nn.Module):
```

```
    def __init__(self, description, embed_dim, share_mlp_dims, output_mlp_dims, dropout):
        super().__init__()
        self.features = {name: (size, type) for name, size, type in description if (type in ["ctn", 'se
        self.build(embed_dim, share_mlp_dims, output_mlp_dims, dropout)
```

```
    def build(self, embed_dim, share_mlp_dims, output_mlp_dims, dropout):
        self.emb_layer = torch.nn.ModuleDict()
```

```

self.ctn_emb_layer = torch.nn.ParameterDict()
self.ctn_linear_layer = torch.nn.ModuleDict()
embed_output_dim = 0
for name, (size, type) in self.features.items():
    if type == 'spr':
        self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
        embed_output_dim += embed_dim
    elif type == 'ctn':
        self.ctn_linear_layer[name] = torch.nn.Linear(1, 1, bias=False)
        embed_output_dim += 1
    elif type == 'seq':
        self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
        embed_output_dim += embed_dim
    else:
        raise ValueError('unkown feature type: {}'.format(type))

self.share_mlp = MultiLayerPerceptron(embed_output_dim, share_mlp_dims, dropout, output_layer=F

hidden_dim = share_mlp_dims[-1]# + 1

# 指数分布参数分支 (快滑峰)
self.lambda_layer = nn.Sequential(
    nn.Linear(hidden_dim, 1),
    nn.Softplus(beta=0.5)
)

comp_num = 10
self.mixture_logits = nn.Linear(hidden_dim, comp_num+1)
self.gauss_mu = nn.Sequential(
    nn.Linear(hidden_dim, comp_num),
    # MultiLayerPerceptron(hidden_dim, output_mlp_dims, dropout, output_layer=True),
    nn.Softplus()
)

```

Q: What is EGMN and what responsibility does it have? **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What are the important methods on EGMN? **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```

__init__
build
init
forward
loss
get_quantile
predict

```

Q: What does the implementation look like for EGMN? **A:** Excerpt from the file:

```

class EGMN(torch.nn.Module):

    def __init__(self, description, embed_dim, share_mlp_dims, output_mlp_dims, dropout):

```

```

super().__init__()
self.features = {name: (size, type) for name, size, type in description if (type in ["ctn", 'seq', 'spr'])}
self.build(embed_dim, share_mlp_dims, output_mlp_dims, dropout)

def build(self, embed_dim, share_mlp_dims, output_mlp_dims, dropout):
    self.emb_layer = torch.nn.ModuleDict()
    self.ctn_emb_layer = torch.nn.ParameterDict()
    self.ctn_linear_layer = torch.nn.ModuleDict()
    embed_output_dim = 0
    for name, (size, type) in self.features.items():
        if type == 'spr':
            self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
            embed_output_dim += embed_dim
        elif type == 'ctn':
            self.ctn_linear_layer[name] = torch.nn.Linear(1, 1, bias=False)
            embed_output_dim += 1
        elif type == 'seq':
            self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
            embed_output_dim += embed_dim
        else:
            raise ValueError('unkown feature type: {}'.format(type))

    self.share_mlp = MultiLayerPerceptron(embed_output_dim, share_mlp_dims, dropout, output_layer=False)

    hidden_dim = share_mlp_dims[-1]# + 1

    # 指数分布参数分支 (快滑峰)
    self.lambda_layer = nn.Sequential(
        nn.Linear(hidden_dim, 1),
        nn.Softplus(beta=0.5)
    )

    comp_num = 10
    self.mixture_logits = nn.Linear(hidden_dim, comp_num+1)
    self.gauss_mu = nn.Sequential(
        nn.Linear(hidden_dim, comp_num),
        # MultiLayerPerceptron(hidden_dim, output_mlp_dims, dropout, output_layer=True),
        nn.Softplus()
    )
    self.gauss_sigma = nn.Sequential(
        nn.Linear(hidden_dim, comp_num),
        # MultiLayerPerceptron(hidden_dim, output_mlp_dims, dropout, output_layer=True),
        nn.Softplus()
    )
    return

def init(self):
    for param in self.parameters():
        torch.nn.init.uniform_(param, -0.01, 0.01)

def forward(self, x_dict):
    linears = []
    embs = []
    for name, (_, type) in self.features.items():
        x = x_dict[name]
        if type == 'spr':
            embs.append(self.emb_layer[name](x).squeeze(1))
        elif type == 'ctn':
            linears.append(self.ctn_linear_layer[name](x))
        elif type == 'seq':
            seq_emb = self.emb_layer[name](x)
            seq_mask = torch.unsqueeze(x_dict["{}mask".format(name)], dim=2)
            embs.append(torch.sum(seq_emb * seq_mask, dim=1) / torch.sum(seq_mask, dim=1))
        else:
            raise ValueError('unkwon feature: {}'.format(name))
    emb = torch.concat(embs + linears, dim=1)

```

```

hidden = self.share_mlp(emb)
# hidden = torch.concat([hidden, engage_pred.view(-1, 1)], dim=1)

lambda_ = self.lambda_layer(hidden) + 1e-6
pi = self.mixture_logits(hidden) # [batch, componet]

mu = self.gauss_mu(hidden) + 1/lambda_ # [batch, component]
# mu = torch.cumsum(mu, dim=1) + 1/lambda_
sigma = self.gauss_sigma(hidden) + 1e-6 # [batch, component]

```

Q: Why is EGMN needed by training scripts? **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

Q: Example: end-to-end data flow involving this file **A:** Core pipeline shape (schematic):

```

for features, label in dataloaders["train"]:
    out = model(features)
    loss = ...
    loss.backward(); optimizer.step()

```

Q: Full source code of the Python file **A:** The complete file is included verbatim for reference.

```

import torch
import torch.nn as nn
import torch.distributions as D
import torch.nn.functional as F
import numpy as np

from model.layers import FactorizationMachine, MultiLayerPerceptron, DurationMultiLayerPerceptron

class EGMN(torch.nn.Module):

    def __init__(self, description, embed_dim, share_mlp_dims, output_mlp_dims, dropout):
        super().__init__()
        self.features = {name: (size, type) for name, size, type in description if (type in ["ctn", 'seq', 'spr'])}
        self.build(embed_dim, share_mlp_dims, output_mlp_dims, dropout)

    def build(self, embed_dim, share_mlp_dims, output_mlp_dims, dropout):
        self.emb_layer = torch.nn.ModuleDict()
        self.ctn_emb_layer = torch.nn.ParameterDict()
        self.ctn_linear_layer = torch.nn.ModuleDict()
        embed_output_dim = 0
        for name, (size, type) in self.features.items():
            if type == 'spr':
                self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
                embed_output_dim += embed_dim
            elif type == 'ctn':
                self.ctn_linear_layer[name] = torch.nn.Linear(1, 1, bias=False)
                embed_output_dim += 1
            elif type == 'seq':
                self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
                embed_output_dim += embed_dim
            else:
                raise ValueError('unkown feature type: {}'.format(type))

        self.share_mlp = MultiLayerPerceptron(embed_output_dim, share_mlp_dims, dropout, output_layer=False)

        hidden_dim = share_mlp_dims[-1] # + 1

```

```

# 指数分布参数分支 (快滑峰)
self.lambda_layer = nn.Sequential(
    nn.Linear(hidden_dim, 1),
    nn.Softplus(beta=0.5)
)

comp_num = 10
self.mixture_logits = nn.Linear(hidden_dim, comp_num+1)
self.gauss_mu = nn.Sequential(
    nn.Linear(hidden_dim, comp_num),
    # MultiLayerPerceptron(hidden_dim, output_mlp_dims, dropout, output_layer=True),
    nn.Softplus()
)
self.gauss_sigma = nn.Sequential(
    nn.Linear(hidden_dim, comp_num),
    # MultiLayerPerceptron(hidden_dim, output_mlp_dims, dropout, output_layer=True),
    nn.Softplus()
)
return

def init(self):
    for param in self.parameters():
        torch.nn.init.uniform_(param, -0.01, 0.01)

def forward(self, x_dict):
    linears = []
    embs = []
    for name, (_, type) in self.features.items():
        x = x_dict[name]
        if type == 'spr':
            embs.append(self.emb_layer[name](x).squeeze(1))
        elif type == 'ctn':
            linears.append(self.ctn_linear_layer[name](x))
        elif type == 'seq':
            seq_emb = self.emb_layer[name](x)
            seq_mask = torch.unsqueeze(x_dict["{}_mask".format(name)], dim=2)
            embs.append(torch.sum(seq_emb * seq_mask, dim=1) / torch.sum(seq_mask, dim=1))
        else:
            raise ValueError('unkwon feature: {}'.format(name))
    emb = torch.concat(embs + linears, dim=1)

    hidden = self.share_mlp(emb)
    # hidden = torch.concat([hidden, engage_pred.view(-1, 1)], dim=1)

    lambda_ = self.lambda_layer(hidden) + 1e-6
    pi = self.mixture_logits(hidden) # [batch, componet]

    mu = self.gauss_mu(hidden) + 1/lambda_ # [batch, component]
    # mu = torch.cumsum(mu, dim=1) + 1/lambda_
    sigma = self.gauss_sigma(hidden) + 1e-6 # [batch, component]

    return pi, lambda_, mu, sigma

def loss(self, y_true, pi, lambda_, mu, sigma, duration):
    batch_size = y_true.shape[0]
    y_true = y_true.view(-1, 1)

    # 指数分布 (快滑峰)
    exp_dist = D.Exponential(rate=lambda_.view(-1))
    log_prob_short = exp_dist.log_prob(y_true.view(-1)).view(batch_size, 1)

    # 高斯分布
    log_prob_all = []
    for comp_idx in range(mu.shape[1]):
        normal_dist = D.Normal(loc=mu[:, comp_idx], scale=sigma[:, comp_idx])
        trunc_min = torch.zeros_like(mu[:, comp_idx])
        prob_long = 1.0 - normal_dist.cdf(trunc_min) # 左侧 <0截断

```

```

        log_prob = normal_dist.log_prob(y_true.view(-1)) - torch.log(prob_long + 1e-6) # [batch, comp_nm]
        log_prob_all.append(log_prob.view(-1, 1))
    log_prob_all = torch.concat([log_prob_short] + log_prob_all, dim=1)

    # 混合概率
    mix_probs = torch.softmax(pi, dim=1)

    # sample_w = (1 + y_true * duration)
    # sample_w = torch.where(y_true * video_durations.view(-1, 1) < 0.005, 0.5 * torch.ones_like(y_true), torch.ones_like(y_
    # nll loss
    log_mix_probs = torch.log_softmax(pi, dim=1)
    total_log_prob = torch.logsumexp(
        log_mix_probs + log_prob_all,
        dim=1, keepdim=True
    )
    nll_loss = -torch.mean(total_log_prob)

    # reconstruction loss
    pi = torch.softmax(pi, dim=1)
    pred = torch.sum(pi * torch.concat([1/lambda_, mu], dim=1), dim=1, keepdim=True)
    reg_loss = F.l1_loss(pred, y_true.float())

    # mixture entropy loss
    entropy_loss = torch.sum(mix_probs * torch.log(mix_probs + 1e-6), dim=1).mean()

    return nll_loss, reg_loss, entropy_loss

def get_quantile(self, pi, lambda_, mu, sigma, tau=0.5):
    exp_dist = D.Exponential(rate=lambda_.view(-1, 1))
    norm_dist_list = []
    for comp_idx in range(mu.shape[1]):
        normal_dist = D.Normal(loc=mu[:, comp_idx:comp_idx+1], scale=sigma[:, comp_idx:comp_idx+1])
        norm_dist_list.append(normal_dist)
    try_list = torch.arange(0, 1, 0.0001).view(1, -1).to(pi.device)
    cdf = exp_dist.cdf(try_list.view(1, -1))
    for norm_dist in norm_dist_list:
        cdf += norm_dist.cdf(try_list.view(1, -1))
    try_list = try_list.view(-1)
    idx = (cdf < tau).to(torch.int8).sum(dim=1)
    return try_list[idx]

def predict(self, x):
    with torch.no_grad():
        pi, lambda_, mu, sigma = self.forward(x)
        pi = torch.softmax(pi, dim=1)
        return torch.sum(pi * torch.concat([1/lambda_, mu], dim=1), dim=1)

```